

PROGRAMMING II

JAVA VARIABLES AND CALL STACK

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026

KINDS OF VARIABLE

x_1

PROGRAM MEMORY

Static memory: used for elements shared by all class instances

- ▶ Like class definitions and **static** fields

Heap memory: used for elements reserved to class instances

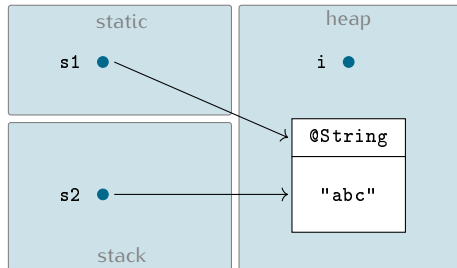
- ▶ Like objects created with **new**
- ▶ Also instance fields are stored in the heap, at object creation

Stack memory: used for elements created at runtime

- ▶ Like local variables and method parameters

KINDS OF PROGRAM MEMORY (CONT'D)

```
public classClazz {
    int i;
    static String s1;
    void aMethod() {
        String s2;
        s2 = new String("abc");
        s1 = s2;
    }
}
```



INSTANCE VARIABLES

Instance variables are stored in the heap

- ▶ Also called class fields

There is a different copy in each instance of the class

- ▶ Created and initialized whenever an object is created
- ▶ Always (implicitly) initialized

// no compilation error

Instance variables are **not shared** between objects from the same class

LOCAL VARIABLES

Local variables are stored in the stack

- ▶ Defined within methods (parameters) or code blocks

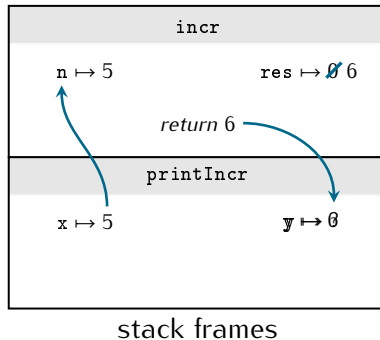
There is a different copy in each method or code block

- ▶ Created at the beginning of the method or code block
- ▶ Automatically destroyed at the end of the method or code block

Instance variables follow block **scope** visibility *// no nested scope shadowing*

STACK FRAME

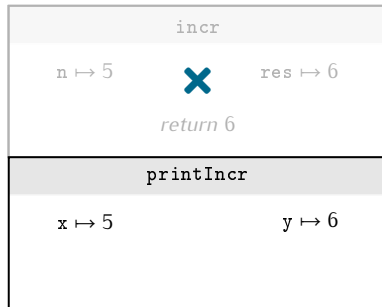
- `int incr(int n) {`
 - `int res = 0;`
 - `res = n + 1;`
 - `return res;`
- `void printIncr(int x) {`
 - `int y = incr(x);`
 - `System.out.println(y);`



Here `n` is the **formal parameter** (variable), while 5 is the **actual parameter** (value)

STACK FRAME (CONT'D)

```
int incr(int n) {  
    int res = 0;  
    res = n + 1;  
    return res;  
}  
  
void printIncr(int x) {  
    int y = incr(x);  
    ● System.out.println(y);  
}
```



stack frames

Here `n` is the **formal parameter** (variable), while 5 is the **actual parameter** (value)

STATIC VARIABLES

Static variables are stored in the static memory

- ▶ Defined as `static` fields or constants

The same copy is shared by all instances of a class

- ▶ Created at class loading

Static variables can be concurrently accessed by objects from the same class

STATIC CODE BLOCK

Static variables are initialized during declaration with single values

- Multi-statement initialization is allowed by using **static code blocks**

```
static int[] arr; // this must go first      1
                                              2
static {                                     3
    arr = new int[3];                        4
    arr[0] = 1;                             5
    arr[1] = 2;                             6
    arr[2] = 0;                             7
}                                              8
```

Static fields and blocks are resolved by the JVM in the order of their declaration

GARBAGE COLLECTION

The garbage collector automatically releases unused memory

- ▶ It periodically checks the memory looking for inactive references
- ▶ It does not affect static and instance variables
- ▶ One can wake up the garbage collector with `System.gc()` *// no guarantees*

```
public static void main(String[]) {  
    Car myCar = new Car();  
    myCar = null;  
    System.gc(); // the object is probably de-allocated  
}
```

1
2
3
4
5

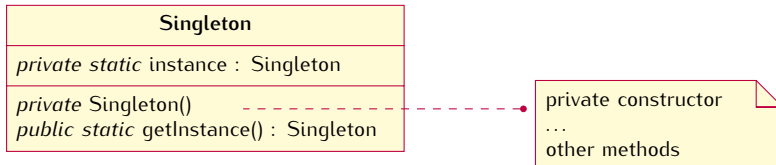
SINGLETON PATTERN

Context: A class represents a concept that requires a single instance

Problem: Clients could use this class in an inappropriate way

Solution: Force to have a single class instance in memory (singleton)

SINGLETON PATTERN (CONT'D)



Restrict class instantiation to a single object instance

The class has a method returning the single object

SINGLETON CLASS

```
public RingOfPower {                                1
    private static String owner = "Sauron";         2
    private static RingOfPower instance;            3

                                                    4
    private RingOfPower() { }                       5
                                                    6

    public static RingOfPower getInstance() {        7
        if (instance == null) { instance = new RingOfPower(); } 8
        return instance;                            9
    }                                                10
                                                    11

    public String getOwner() { return owner; }       12
}                                                    13
```

To use the class: `RingOfPower.getInstance()`

IMMUTABLE OBJECTS



IMMUTABILITY

Immutability: objects internal state remains constant after creation

- The classic example is the String class

```
String name = "Samuel";           1
String newName = name.replace("uel", "");    2
                                     3
System.out.println(newName); // the output will be 'Sam' 4
System.out.println(name);    // the output will be 'Samuel' 5
```

Immutable classes provides **read-only** methods

IMMUTABILITY (CONT'D)

The **final** keyword allows to freeze a variable with its initialization value

- ▶ After **final** `Person sam = new Person("Sam");` the value of `sam` cannot change

But, only the reference cannot change, the internal state of the object can

```
final Person sam = new Person("Sam");           1
                                                    2
sam = new Person("Joe"); // not allowed, the reference cannot change 3
sam.name = "Joe"; // allowed                      4
```

IMMUTABLE OBJECTS

Make **final** the fields of an immutable object

- For instance, the field `name` of the `Person` class

Initialize the fields inside class constructors

```
class Person {  
    private final String name; // initialization in the constructor  
  
    Person(String name) { this.name = name; }  
  
    public String getName() { return name; } // name getter allowed  
  
    // name setter not allowed  
}
```

1
2
3
4
5
6
7
8
9

MUTABILITY AND IMMUTABILITY

	Mutable Objects	Immutable Objects
Modifiability	Can be changed after creation	Remain constant once created
Thread Safety	May require synchronization	Inherently thread-safe
Predictability	Less understandable	Easier to debug
Performance	Generally slower	Generally more efficient

ALIASING AND MUTABILITY

Consider the mutable implementation of Person

```
Person sam = new Person("Sam");           1
Person joe = sam; // joe and sam point to the same object 2
                                                    3
joe.name = "Joe";                             4
                                                    5

System.out.println(sam + " : " + joe); // outputs 'Joe : Joe' 6
                                                    7
```

Side-effect: when we modify joe object we also modify sam object

ALIASING AND MUTABILITY (CONT'D)

Consider the immutable implementation of Person

```
Person sam = new Person("Sam");           1
Person joe = sam; // joe and sam point to the same object 2
                                                    3
joe.name = "Joe"; // compiler error          4
joe = new Person("Joe");                    5
                                                    6
System.out.println(sam + "␣:␣" + joe); // outputs 'Sam : Joe' 7
```

No side-effects

PASSING ARGUMENTS TO METHODS

How parameters are passed from the caller to the callee?

Pass-by-value

```
class Clazz { // actual parameter is copied into formal parameter 1
    void aMethod() { // caller 2
        int x = 5; 3
        int y = increment(x); 4
        System.out.println(x + " : " + y); 5
    } 6
    int increment(int n) { // callee 7
        return ++n; 8
    } 9
} // x and n are different variables 10 11 12 13 14
```

actual parameter

'5 : 6' is printed here

formal parameter

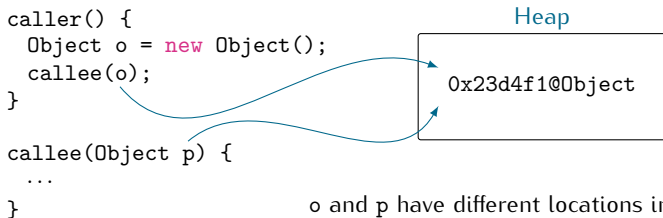
JAVA ARGUMENT PASSING

Java **always** use pass-by-value

- Fine for primitive variables, but tricky for object variables

The value of an object variable is a reference, the value copied is a reference

- Actual and formal parameter points to the same object in memory



TRICKY ARGUMENT PASSING

```
class Clazz { // pass-by-value      1
    void aMethod() { // caller      2
        Person samuel = new Person("Samuel");      3
        Person sam = shortenName(samuel);      4
        System.out.println(samuel + "␣:␣" + sam);      5
    }      6
    Person shortenName(Person p) { // callee      7
        p.name = p.name.substring(0, 3);      8
        return new Person(p.name);      9
    }      10
}      11
      12
      13
      14
      15
      16
```

'Sam : Sam' is printed


```

class Clazz { // pass-by-value
    void aMethod() { // caller
        Person samuel = new Person("Samuel");

        Person sam = shortenName(samuel);

        System.out.println(samuel + "␣:␣" + sam); 'Samuel : Sam' is printed
    }

    Person shortenName(Person p) { // callee
        p = new Person(p.name);
        p.name = p.name.substring(0, 3);
        return p;
    }
}

```

WRAPPER CLASSES



PURE OBJECT ORIENTATION

In a pure OO language, there are only classes and objects

- ▶ For the sake of efficiency, Java use primitive types (`int`, `float`, ...)

Wrapper classes are object versions of the primitive types

- ▶ They define conversion operations between different types

WRAPPER CLASSES

Primitive type	Wrapper class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Char
boolean	Boolean

Wrapper class constructors create objects that contain a primitive value

- ▶ Wrapper classes are defined in the `java.lang` package

WRAPPER CLASSES (CONT'D)

To wrap a `int` value

▶ `Integer wi = new Integer(3);`

// boxing

▶ To unwrap it: `int i = wi.intValue();`

// unboxing

To wrap a `double` value

▶ `Double wd = new Double(2.1);`

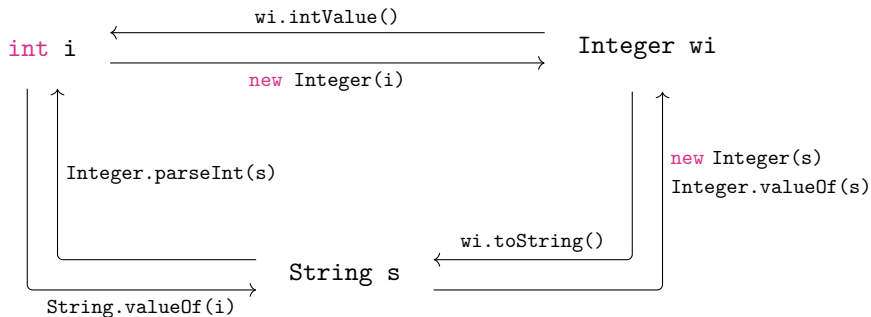
// boxing

▶ To unwrap it: `double d = wd.doubleValue();`

// unboxing

Here `wi` and `wd` are references to objects containing primitive values

CONVERSIONS



AUTOBOXING

Since Java 5, the compiler performs automatic boxing and unboxing

<code>Integer wi = new Integer(2);</code>	1
<code>int j = 0;</code>	2
	3
<code>j = wi + 5; // instead of j = wi.intValue() + 5;</code>	4
	5
<code>wi = 3; // instead of wi = new Integer(3);</code>	6
	7
<code>wi = j; // instead of wi = new Integer(j);</code>	8

Indeed, `new Integer()` is deprecated since Java 9

UTILITY METHODS

Wrapper classes come with utility methods

The Character class provides some useful **static** methods

- ▶ `isLetter()`, `isDigit()`, `isSpaceChar()`, ...
- ▶ `toUpperCase()`, `toLowerCase()`, ...

```
Character wc1 = 'g';           // using "g" will cause a compilation error    1
Character wc2 = Character.toUpperCase(wc1);                                   2
                                                                           3
System.out.println(wc2); // the output will be 'G'                           4
System.out.println(wc1); // the output will be 'g'                           5
```


LIKE A STRING

Wrapper classes are just value containers, like the String class

- ▶ Wrapper class objects are immutable, like String objects
- ▶ Wrapper classes come with a toString() method

```
int j = 3;                                     1
Integer wi = j; // a new '3' Integer object is created, wi points to it 2
wi = 7;      // a new '7' Integer object is created, wi points to it 3
```

PITFALLS

Wrapper classes comparison with == checks reference equality

- Use the built-in equals() method

```

int x = 3;                                     1
Integer wx = x;                                2
int y = 3;                                     3
Integer wy = y;                                4
Integer wi = new Integer(3);                    5
                                                6

System.out.println(x == y);                    7
System.out.println(wx == wi);                  8
System.out.println(wx == wy);                  9
System.out.println(wx.equals(wy));             10
System.out.println(wx.equals(wi));             11
    
```

PITFALLS (CONT'D)

The method `Integer.parseInt()` takes a string and returns its numeric value

- The string must represent a valid integer value

```
int i = Integer.parseInt("13") + 3; // the output will be '16'      1
int j = Integer.parseInt("␣13");    // the JVM will throw an exception 2
int k = Integer.parseInt("2.1");     // the JVM will throw an exception 3
```

Q + A

